

Resumo — Protocolo HTTP

Programação Web · Material de apoio para a Aula 1

1. O que é o HTTP

HTTP (HyperText Transfer Protocol) é o protocolo de comunicação usado pela Web para troca de mensagens entre cliente (navegador) e servidor. É um protocolo da camada de aplicação, baseado no modelo requisição-resposta: o cliente envia uma requisição, o servidor devolve uma resposta.

- **Stateless:** HTTP é stateless (sem estado): cada requisição é independente — o servidor não guarda memória de requisições anteriores por padrão.
- **Camada de transporte:** ele é executado sobre TCP (na maioria dos casos), que garante a entrega confiável dos dados.

2. Estrutura de uma requisição HTTP

Toda requisição HTTP tem três partes:

- **Linha de requisição:** método + caminho (URL) + versão do protocolo.
- **Headers:** metadados sobre a requisição (tipo de conteúdo aceito, autenticação, etc.).
- **Corpo (opcional):** dados enviados junto (comum em POST/PUT — ex: dados de um formulário).

```
GET /produtos/lista?categoria=livros HTTP/1.1
Host: www.escola.com.br
User-Agent: Mozilla/5.0
Accept: text/html
```

3. Principais métodos HTTP

Método	Uso típico
GET	Buscar/ler um recurso (não deve alterar dados no servidor).
POST	Criar um novo recurso (ex: enviar um formulário, cadastrar algo).
PUT	Atualizar um recurso por completo, substituindo o existente.
PATCH	Atualizar um recurso parcialmente.
DELETE	Remover um recurso.
HEAD	Como o GET, mas só pede os headers (sem corpo) — útil pra checar se um recurso existe.

4. Estrutura de uma resposta HTTP

- **Linha de status:** versão do protocolo + código de status + texto descritivo.
- **Headers:** metadados da resposta (tipo do conteúdo, tamanho, cache, etc.).
- **Corpo:** o conteúdo retornado (HTML, JSON, imagem, etc.).

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 1274

<html>...</html>
```

5. Códigos de status HTTP

Faixa	Significado	Exemplos
1xx	Informativo	100 Continue
2xx	Sucesso	200 OK · 201 Created · 204 No Content
3xx	Redirecionamento	301 Moved Permanently · 302 Found · 304 Not Modified
4xx	Erro do cliente	400 Bad Request · 401 Unauthorized · 403 Forbidden · 404 Not Found
5xx	Erro do servidor	500 Internal Server Error · 502 Bad Gateway · 503 Service Unavailable

6. Headers comuns

Header	Pra que serve
Content-Type	Formato do conteúdo enviado/recebido (text/html, application/json...).
Authorization	Credenciais de autenticação (ex: token de acesso).
Cookie / Set-Cookie	Envio e definição de cookies entre cliente e servidor.
Cache-Control	Regras de cache (por quanto tempo guardar a resposta, etc.).
User-Agent	Identifica o navegador/cliente que fez a requisição.
Accept	Quais formatos de resposta o cliente aceita receber.

7. HTTP é stateless — e a sessão do usuário?

Como o HTTP não guarda memória entre requisições, aplicações web usam mecanismos extras pra simular "sessão" (ex: manter o usuário logado):

- **Cookies:** um pequeno dado guardado no navegador e reenviado a cada requisição pro mesmo domínio.
- **Sessão no servidor:** o servidor guarda dados da sessão e associa a um ID enviado via cookie.

- **Tokens (ex: JWT):** um token que carrega as informações do usuário, sem precisar de estado no servidor (comum em APIs).

Isso vai aparecer com mais detalhe quando começarmos a mexer com autenticação em Flask.

8. Evolução do protocolo (rapidamente)

- **HTTP/1.1:** versão mais usada historicamente; conexões podem ser reaproveitadas (keep-alive).
- **HTTP/2:** multiplexação de requisições numa mesma conexão, mais rápido.
- **HTTP/3:** roda sobre QUIC (UDP) em vez de TCP, ainda mais rápido em redes instáveis.

9. E o HTTPS?

HTTPS é o HTTP rodando sobre uma camada de criptografia (TLS). O protocolo e as mensagens trocadas são as mesmas — a diferença é que o conteúdo fica protegido contra interceptação no caminho entre cliente e servidor. Hoje é o padrão esperado para qualquer site.